

# 2025 年全国大学生电子设计竞赛

## 简易自行瞄准装置 (E 题)

### 【本科组】



2025 年 8 月 2 日

## 摘 要

本项目为“简易自行瞄准装置”。该系统以 MSPM0G3507 微控制器为核心，负责控制一个二维云台和智能小车。视觉部分采用 K230 模块，其内置的人工智能加速器能高效识别目标，并将处理后的数据发送给主控核心。

智能小车利用灰度传感器循线，并通过 PD 算法和陀螺仪进行速度与姿态控制，以实现精确定位和移动。瞄准模块通过 K230 实时处理摄像头采集的图像，识别目标靶的黑框与靶心，随后控制装载步进电机的二维云台，引导激光笔完成精确瞄准和轨迹巡线。系统还设计了基于触摸交互的 UI 界面，可脱机调节图像识别的灰度阈值。

测试结果显示，寻迹小车和瞄准模块均表现出良好的精度，距离误差较小。

**关键词：**K230；PID 算法；MSPM0G3507

# 一、系统方案设计与论证

本系统主要由电源模块，总控制模块，K230 识别模块，二维电控云台组成，其系统框图如下图所示：

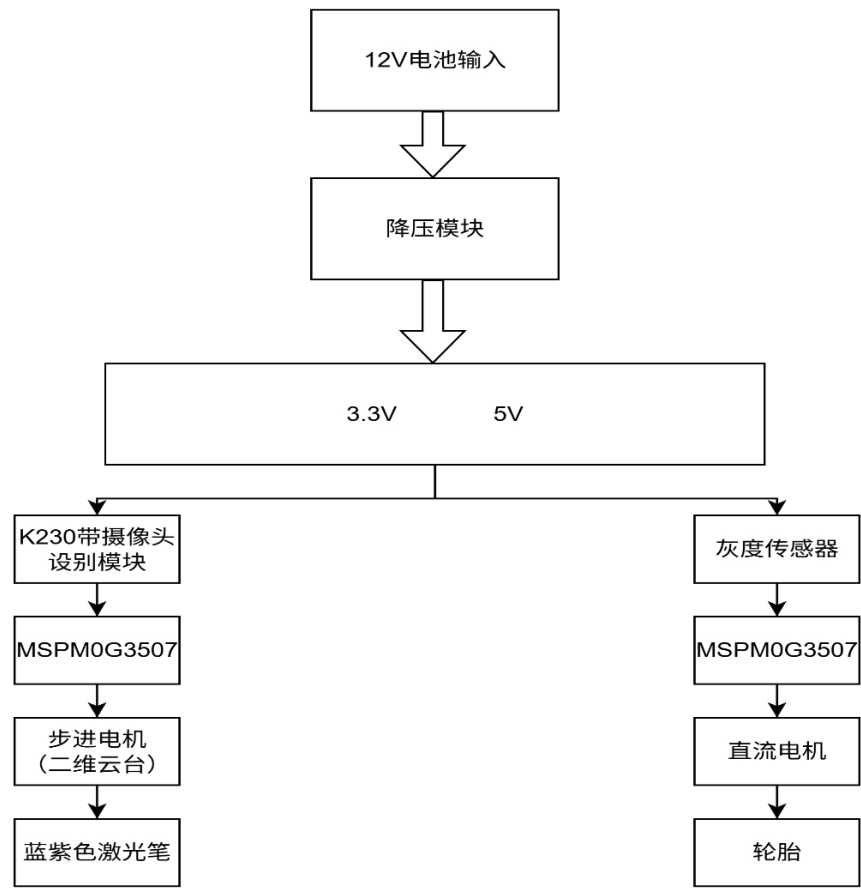


图 1.1 系统框图

## 1.1 控制系统的方案论证与选择

方案一：MSPM0L1306

MSPM0L1306 是一款基于 Arm® Cortex®-M0+ 内核的入门级 32 位微控制器。。然该微控制器的主频相对较低，片上集成的 SRAM 和闪存容量也有限，在处理需要密集计算和快速响应的复杂算法时，其性能和存储空间可能会成为瓶颈。

方案二：MSPM0G3507

MSPM0G3507 同样是一款基于 Arm® Cortex®-M0+ 内核的 32 位微控制器，但其定位更高，性能也更为强大。

**结论：**综合以上两种方案，我们选择运算速度快、内存空间充裕且外设性能优越的 MSPM0G3507 作为核心。

## 1.2 视频识别模块的方案论证与选择

### 方案一：OPENMV4 PLUS

OPENMV4 PLUS 是一款集成化机器视觉模块。。基于微控制器的 OPENMV4 PLUS 在处理高速运动带来的图像畸变、进行复杂坐标解算以及保证低延迟控制方面，其算力可能面临挑战。

### 方案二：K230 视觉模块

K230 采用了双核 RISC-V 架构，并内置了新一代的 KPU（Kendryte Process Unit）人工智能加速器。这使其拥有了远超普通微控制器的强大算力，能够以更高的帧率和更低的延迟运行复杂的视觉算法。对于本装置而言，K230 能轻松实时解算出目标相对于车体的精确方位，从而精准控制云台，确保光斑始终稳定。

**结论：**综合以上两种方案，需要选择处理能力强、延迟低的 K230。

## 1.3 电机论证与选择

小车：310 直流减速电机能提供高转速和高扭矩，通过 PWM 信号即可快速响应，满足赛题对圈速的要求。

云台：步进电机能通过脉冲信号进行精确的角度控制和位置保持，是实现高精度瞄准和复杂轨迹的理想选择。

## 二、理论分析与计算

### 2.1 自动巡迹控制分析

#### 2.1.1 灰度传感器

小车的寻迹由装载在车头的灰度传感器决定，通过不同权重通过 PD 算法控制两轮车速以实现小车的巡线控制。

#### 2.1.2 PID 控制分析

PID 通过三系数调节动态改变施加到被控对象的输出值（Out），使得被控对象某一物理量的实际值（Actual），能够快速、准确、稳定地跟踪到指定的目标值（Target）其基于误差（Error）调控，目的使误差始终为 0

定义误差： $error(t) = target(t) - actual(t)$

$$\text{PID 输出值: } out(t) = K_p(error(t) + \frac{1}{T_i} \int_0^t error(t) dt + \frac{T_d derror(t)}{dt})$$

$$\text{PID 输出值: } out(t) = K_p * error(t) + K_i * \int_0^t error(t) dt + K_d * \frac{derror(t)}{dt}$$

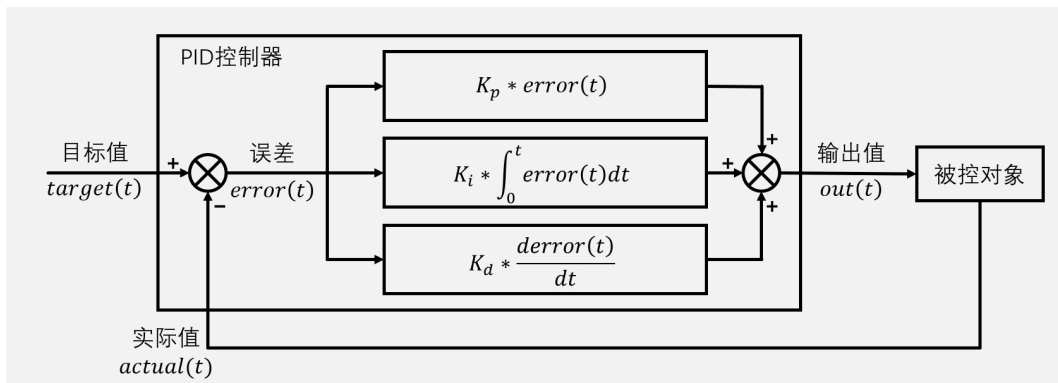
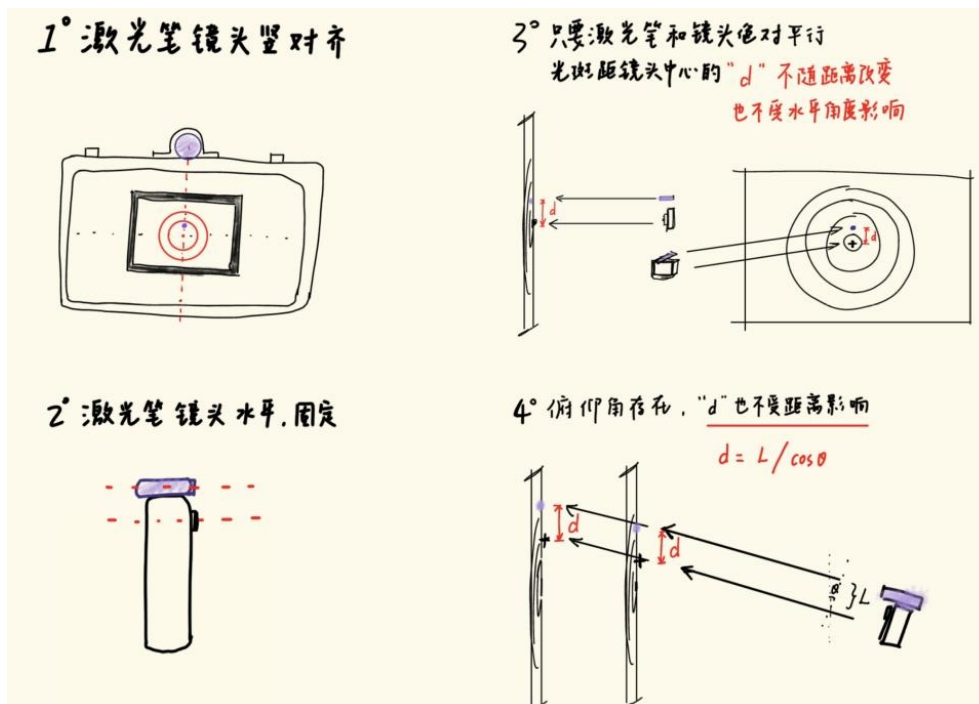


图 2.1 PID 控制框图

## 2.2 图像识别方法的分析

系统的视觉识别模块基于 K230 开发板实现，主要功能包括目标靶的黑框识别、靶心定位以及圆环轨迹定位。模块通过摄像头实时采集图像，经过图像处理和几何计算，确定目标位置，并将坐标差通过串口发送给主控系统，控制激光笔精准瞄准。

如图所示，激光笔与摄像头刚性固定，只要初始俯仰角度确定好后，始终保持该角度不变，则激光射点总是打在镜头中心正上方固定高度处，瞄准时只需将准心向下相应高度进行瞄准，射出激光即可正中靶心。



### 2.2.1 矩形确认机制

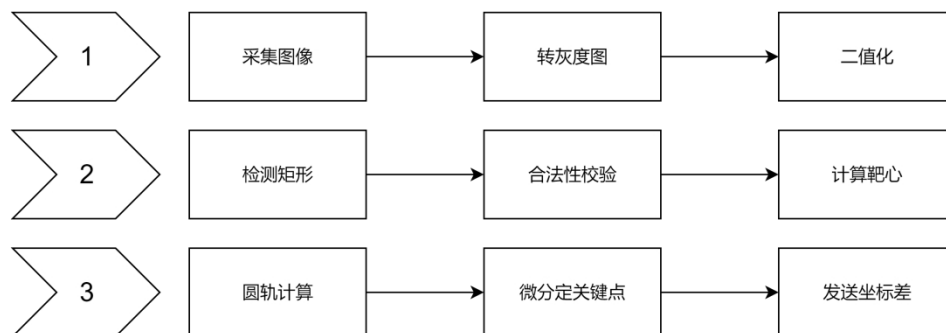
1. 双矩形匹配：仅当图像中同时检测到一大一小矩形时，才认为目标有效。
2. 中心距离约束：两个矩形的中心距离需小于阈值避免误匹配环境噪点。
3. 面积排序：对检测到的矩形按面积降序排列，优先处理面积最大的。

### 2.2.2 6cm 圆环循线算法

1. 比例计算：根据黑框的实际尺寸（除去黑胶带后尺寸为：17.6\*26.3）与图像中的像素尺寸，计算比例系数：
$$\text{ratio} = \frac{\text{width\_pixels}}{17.6} + \frac{\text{height\_pixels}}{26.3}$$

轨迹点生成：以靶心为中心，按半径 6cm 和比例系数计算圆周上的 16 个点，形成连续的轨迹点序列。

2. 云台巡线：按顺序遍历 17 个轨迹点（16 个圆周点+1 个闭合点），通过串口通信传递信息，控制步进电机转向，到达一个目标点位通过连续发送 `(0,0)` 确认到达，立刻按序追寻下一个目标点。



### 2.2.3 脱机调节灰度阈值

针对 K230 的特性进行了专门的 UI 设计，完全基于触摸交互实现，无需外接输入设备，体现了高度集成的设计思想，包含两种模式：

正常识别模式：简洁显示识别状态和系统信息

阈值调节模式：提供完整的交互界面用于参数调整



三、电路与程序设计

3.1 拓展板电路设计

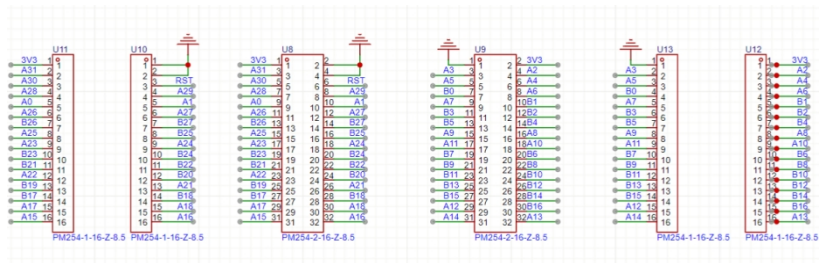


图 3.1 原理图

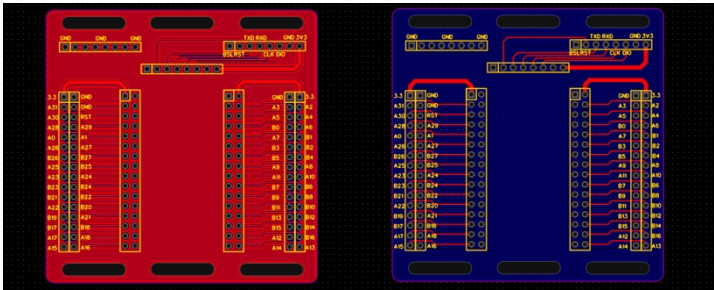


图 3.3 PCB 图

3.2 程序设计

3.2.1 自动寻迹小车基本设计

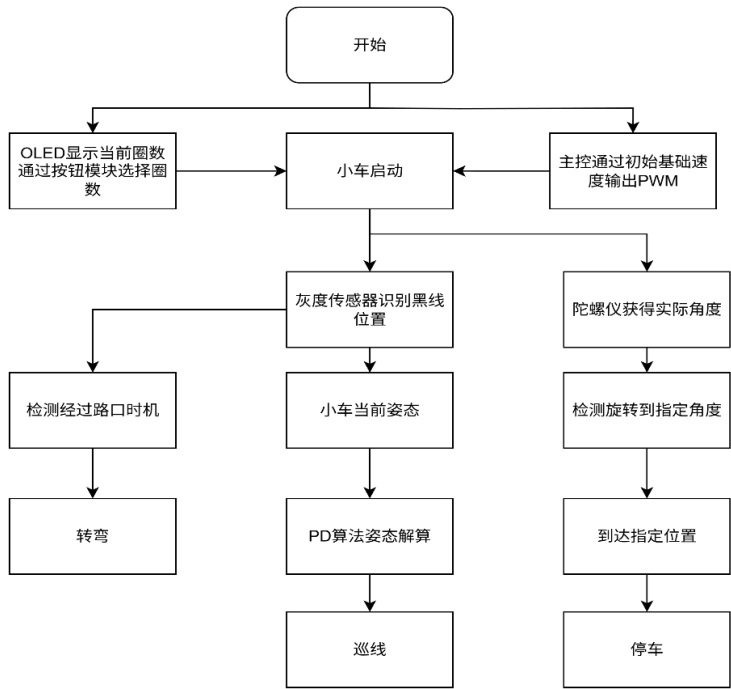


图 3.4 自动寻迹小车流程图

3.2.2 瞄准模块设计

平面瞄准是在云台的一个方向设置一个坐标平面，将平面的位置坐标转化为二维云台的球面坐标，从而达到我想要的位置到云台的姿态解算。

环形瞄准是通过解除平面瞄准的部分限制达到的，可以使云台在周身全向瞄准，通过发现自身的运动改变视野，以发现目标的方法实现跟随与视野外的定位。因此这更适合运动定位。

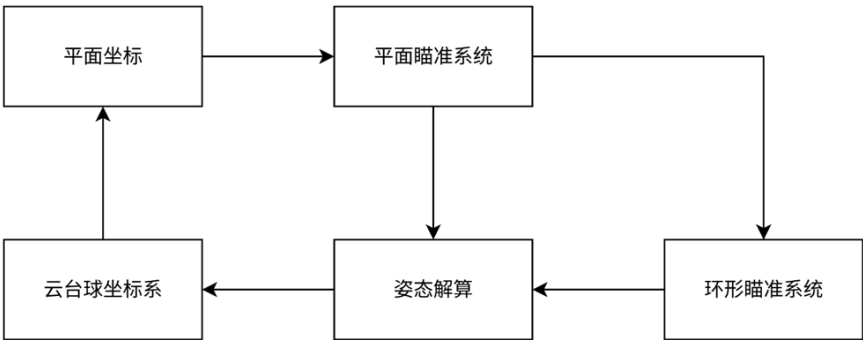


图 3.5 二维云台流程图

四、测试方案与测试结果

4.1 测试方法

对运动控制系统和自动追踪系统分别单独测试，并对其程序进行分块处理。

4.1.1 寻迹小车测试

(1) 运动控制测试：通过给定不同的圈数，测试小车在不同距离下，起始位置与目标位置的偏移，直到小车结束位置投影能够覆盖结束位置投影。

4.1.2 瞄准模块测试

(1) 追踪功能测试：首先让瞄准系统远离靶纸方向，开启打靶功能，测试蓝紫色激光是否能够定点到靶中心，记录误差。

(2) 停止功能测试：使瞄准模块进行靶点打靶，在运动到的瞬间发送串口，使二维云台迅速停止，精准停在靶点位置，记录误差。

4.2 测试结果

4.2.1 寻迹小车测试结果

|      |   |     |     |     |     |
|------|---|-----|-----|-----|-----|
| 测试次数 | 1 | 2   | 3   | 4   | 5   |
| 距离误差 | 1 | 1.5 | 1.3 | 1.1 | 0.8 |



#### 4.2.2 瞄准模块测试结果

|      |     |     |      |     |      |
|------|-----|-----|------|-----|------|
| 测试次数 | 1   | 2   | 3    | 4   | 5    |
| 距离误差 | 0.1 | 0.2 | 0.05 | 0.1 | 0.08 |

|      |   |   |   |   |   |
|------|---|---|---|---|---|
| 测试次数 | 1 | 2 | 3 | 4 | 5 |
| 停止时间 | 0 | 0 | 0 | 0 | 0 |

附件：

### 1. 小车循迹控制代码

```
1. void xunji(void)
2. {
3.   uint8_t sensor_count = 0;
4.   long weighted_sum = 0;
5.   uint8_t i;
6.   for (i = 0; i < 8; i++) {
7.     if (digital[i] < boundary) {
8.       sensor_count++;
9.       weighted_sum += weights[i];
10.    }
11.  }
12.  // --- 容错转向模块 (最后的保险) ---
13.  if (sensor_count == 0) {
14.    PWM_SetCompare(210, 350);
15.    car_turn_left();
16.    while(digital[2] > boundary && digital[3] > boundary && digital[4] > boundary &
        & digital[5] > boundary);
17.    car_go();
18.    PWM_SetCompare(BASE_SPEED, BASE_SPEED);
19.    delay_cycles(4800000);
20.    return;
21.  }
22.  // --- 常规 PD 巡线模块 ---
23.  if (sensor_count > 0) {
24.    error = (float)weighted_sum / sensor_count;
25.    int correction = (int)(Kp * error + Kd * (error - last_error));
26.    last_error = error;
27.    int left_speed = BASE_SPEED - correction;
28.    int right_speed = BASE_SPEED + correction;
29.    if (left_speed > PWM_MAX) left_speed = PWM_MAX;
30.    if (left_speed < 0) left_speed = 0;
31.    if (right_speed > PWM_MAX) right_speed = PWM_MAX;
32.    if (right_speed < 0) right_speed = 0;
33.    PWM_SetCompare(left_speed, right_speed);
34.  }
35. }
```

## 2. 云台控制代码

```
1. void step(struct STEP_MOTOR *step_motor){
2.     if(step_motor->now_steps!= step_motor->target_step){
3.         if (step_motor->target_step > step_motor->now_steps) {
4.             if(step_motor->number == 1) {
5.                 DL_GPIO_setPins(STEP_MOTOR_PORT, STEP_MOTOR_dir1_PIN);
6.                 DL_GPIO_togglePins(STEP_MOTOR_PORT, STEP_MOTOR_st1_PIN);
7.                 step_motor->now_steps++;
8.             }
9.
10.            if(step_motor->number == 2) {
11.                DL_GPIO_setPins(STEP_MOTOR_PORT, STEP_MOTOR_dir2_PIN);
12.                DL_GPIO_togglePins(STEP_MOTOR_PORT, STEP_MOTOR_st2_PIN);
13.                step_motor->now_steps++;
14.            }
15.        }else if(step_motor->target_step < step_motor->now_steps) {
16.            if(step_motor->number == 1) {
17.                DL_GPIO_clearPins(STEP_MOTOR_PORT, STEP_MOTOR_dir1_PIN);
18.                DL_GPIO_togglePins(STEP_MOTOR_PORT, STEP_MOTOR_st1_PIN);
19.                step_motor->now_steps--;
20.            }
21.            if(step_motor->number == 2) {
22.                DL_GPIO_clearPins(STEP_MOTOR_PORT, STEP_MOTOR_dir2_PIN);
23.                DL_GPIO_togglePins(STEP_MOTOR_PORT, STEP_MOTOR_st2_PIN);
24.                step_motor->now_steps--;
25.            }
26.        }
27.
28.    }
29.
30. }
```

## 3. 视觉识别圆环画线代码

```
1. clock.tick()
2.     img = sensor.snapshot()
3.     rects = find_black_frames(img, threshold_adjuster.threshold)
4.
5.     current_valid_center = None
6.     current_target_point = None
7.     rect_detected = False
8.     if len(rects) >= 2:
```

```

9.         rect_detected = True
10.        large_rect, small_rect = rects[0], rects[1]
11.        center_large = calculate_center_from_corners(large_rect.corners())
12.        center_small = calculate_center_from_corners(small_rect.corners())
13.
14.        if center_large and center_small:
15.            if abs(center_large[0] - center_small[0]) < MAX_X_DIFFERENCE:
16.                current_valid_center = center_small
17.                draw_quadrilateral(img, small_rect.corners(), color=(0, 255,
0), thickness=2)
18.
19.                corners = small_rect.corners()
20.                width_pixels = (distance(corners[0], corners[1]) + distance
(corners[2], corners[3])) / 2
21.                height_pixels = (distance(corners[1], corners[2]) + distanc
e(corners[3], corners[0])) / 2
22.
23.                # 计算所有 17 个目标点
24.                all_target_points = calculate_all_target_points(
25.                    current_valid_center,
26.                    width_pixels,
27.                    height_pixels,
28.                    radius_cm=6
29.                )
30.
31.                # 获取当前目标点
32.                if current_target_index < len(all_target_points):
33.                    current_target_point = all_target_points[current_target
_index]
34.
35.                    # 检查是否到达当前目标点
36.                    if current_target_point and current_valid_center:
37.                        dist_to_target = distance(light, current_target_poi
nt)
38.                        if dist_to_target <= TARGET_REACHED_THRESHOLD:
39.                            target_reached = True
40.                        else:
41.                            target_reached = False

```